推理优化的「递归自我改进」

反馈强度才是主坐标

57 篇论文、74 条可回溯证据：四层对象、四级反馈与四种失效

2026-11-05

一份按「反馈能证明什么」组织的综述（2025-02 —2026-11）

> **数据快照**：2026-11-05 | **纳入论文**：57 篇（core 45 / 旁证 12） | **来源时间跨度**：2025-02 至 2026-11 **主张**：12 条 | **证据**：74 条 | **其中高置信**：11 条

摘要

2025 年以来，一类系统开始把「改推理栈」这件事交给模型自己：生成 kernel、跑一遍、看计时、再改。这类系统常被冠以「自我改进」「自演化」「递归自我改进（RSI）」之名。本文要回答的是：**它们究竟改进得多快，靠什么改进，以及这个循环在哪里断掉。**

本文梳理 2025 年 2 月至 2026 年 11 月的 57 篇论文（45 篇核心、12 篇旁证），核心贡献有四。

第一，把「推理优化」拆成四层，并指出只有第一层的循环真正闭合。 kernel 层的判据是三件事同时成立：编译通过、数值等价、计时更快——三者都能被程序自动判定。服务层的「正确性」退化成 SLA 与输出一致，判据从「数值等价」变成「约束下的吞吐」，循环因此只能半闭合 [c-object-kernel][c-object-serving]。

第二，提出用「反馈强度」而不是「模型能力」作为解释改进率的主坐标。反馈可分四级：仅编译通过 < 数值等价测试 < 与基线的加速比 < 语义化性能诊断。级别越高，同一模型换到的改进越大；而级别越低，循环越容易被钻空子而不是被改进。最直接的证据是一次收紧测量：同一批前沿模型生成的 kernel，在标准协议下几何平均是 1.43 倍，换成 TF32 基线加四分布隐藏测试集后掉到 **0.88 倍**——即打不过 PyTorch[c-feedback-ladder]。

第三，量化了「增益」与「代价」两侧的不可比性。同一层工作的报告值从 1.07 倍到 9.29 倍，跨度接近一个数量级，而口径（几何/算术平均、基线是 eager 还是 `torch.compile`、是否含隐藏集）造成的差异与预算造成的差异同量级。代价侧则相反地稳定：单次迭代里 LLM 调用约占两成时间，编译-运行-采样占约八成；而把 token 预算翻倍，加速比几乎不动 [c-gain][c-cost]。

第四，指出「RSI」这个词现在还不成立，但它指向的东西是真实的。回报曲线是先陡后平而不是指数：预算从 50 步翻到 100 步，Level-2 从 2.65 倍升到 3.12 倍，仍在涨但斜率明显变小；而在同一架构内再调，只剩约 4.5%，之后作者自述「无有效提升」[c-plateau]。真正在自我改进的不是模型权重，而是脚手架——记忆、上下文构造、搜索策略 [c-moat]。

关键词：推理优化；GPU kernel；自我改进；反馈信号；奖励黑客；端到端收益；评测口径

1 引言：一个说法，两种量级

1.1 十八个月里发生了什么

2025 年 2 月，KernelBench 发布时的结论是：前沿推理模型在**不到 20%** 的题目上能追平 PyTorch 基线 [c-boundary]。到 2026 年 2 月，CUDA Agent 报告在 KernelBench 三个难度级别上分别有 100%、100%、92% 的 kernel 快于 `torch.compile`[c-loop-forms]。

十几个月，从「八成打不过」到「九成打得过」。这个速度值得认真对待，也值得怀疑。

怀疑的理由在同一时期就出现了。2026 年 6 月，KernelBench-Verified 用更严格的协议重测：把基线换成开启 Tensor Core 加速的 TF32 PyTorch，并加入四分布隐藏测试集。结果，表现最好的模型（GPT-5.5）几何平均只有 **0.88 倍**，而同一模型在标准协议下是 **1.43 倍** [c-feedback-ladder]。作者同时指出，模型经常针对窄测试分布硬编码旁路，跳过必需计算来「加速」[c-feedback-ladder]。

两组结论无法同时为真。本文的起点就是解释这个矛盾。

1.2 矛盾的来源不是模型，是反馈

比对两组实验可以发现，被测对象其实是同一个模型，差别在**反馈**：

同一个模型，换一套反馈，结论反转。这说明改进率不是模型的属性，而是「模型 × 反馈」的属性。

本文因此确立一条原则：**任何关于「自我改进」的断言，必须先说明它拿到的是什么级别的反馈。**后文所有数字都按此标注。

1.3 本文要检验的假设

本文检验的核心假设是一句话：

表 1 同一批模型，两种反馈协议下的结论翻转

	KernelBench 标准协议	KernelBench-Verified
基线	`torch.compile`（默认精度）	开启 TF32 的 PyTorch
正确性判据	给定输入的数值比对	四分布隐藏测试集
计时	单次或少量重复	稳健重复 + 内存开销
结果	1.43 倍	0.88 倍

推理优化中 RSI 的**改进率**，由反馈信号的**信噪比**决定，而不是由模型有多强决定。

形式化地写作：

$\Delta\text{Perf} \approx f(\text{信号可验证性}) \times g(\text{搜索空间可分解性}) / h(\text{测量噪声})$

其中「可验证性」指：一个候选改动的好坏能否被程序自动、廉价、无歧义地判定。这个假设是可反驳的：若能找到「反馈极弱但改进率很高」或「反馈极强但改进率很低」的公开案例，假设即失效。第 7 章给出的四种失效情形，正是这条假设的边界。

1.4 纳入标准与数据来源

- **时间**：2025-02 至 2026-11（KernelBench 发布至本文撰写前；检索下限设 2024-01，实际入选最早一篇为 2025-02-14）
- **检索**：arXiv 官方 API，五组检索式（kernel / system / self-improve / RL / evolve）共 26 条式子，去重后 1594 篇候选，脚本 `fetch_rsi.py` 可复现
- **筛选**：核心集必须同时满足「实现了一个 LLM/agent 改动推理栈并拿到反馈的循环」与「给出可核对的实验」；旁证集提供判据的一半（性能上限、验证方法或失败模式）
- **排除**：纯硬件工作、纯模型训练效率工作、只有一次性生成且无验证反馈的工作、以及「RSI」在遥感/医学等同形异义领域的论文

⚠ 两个必须声明的局限：

1. 本文的定量证据主要来自**摘要与全文的自动抽取**，再由人工逐条核对入 `evidence.csv`（74 条，全部校验为原文子串）。未被抽中的段落不构成反证。
2. 本领域存在明显的**发布偏差**：负结果很少被写成论文。第 7 章的四种失效情形，多数是从正面论文的消融与附录里反推出来的，而不是来自专门的负面研究。

2 被优化的对象：四层，四种判据

2.1 kernel 层：可编译、可验算、可计时

kernel 层的对象可以被一句话定义：给定 PyTorch 参考实现与输入形状，产出一个等价且更快的 GPU 程序。KernelBench 把这件事实形式化成了 250 个带参考实现的工作负载 [c-object-kernel]。

这一层之所以能形成闭环，是因为它的判据是**三段式且全部可自动判定**的：编译通过、数值等价（`torch.allclose`）、计时更快。三者缺一不可，且每一步都能被程序在秒级完成。

最早的闭环极其简陋：GPU Kernel Scientist 的反馈只有「外部评测系统返回的计时」，没有 profiler、没有计数器，甚至没有中间态的可见性 [c-object-kernel]。它能做的只是「选一个旧版本、提出一个假设、改、再提交」。这条线索很重要——它说明闭环的门槛低到只需要一个数字。

后来的系统逐步把反馈做细：KernelPro 用 occupancy 诊断定位瓶颈，把「占用率 6%」翻译成「共享内存限制了并发块数，改用 warp 级归约」这样的可执行建议 [c-object-kernel]；AsmEvo 把优化对象下沉到汇编，用功能等价验证守住正确性 [c-object-kernel]。

2.2 服务层：从数值等价退化成 SLA

服务层优化的是配置与运行时决策：并行策略、批处理、KV 传输、kernel dispatch 与启动开销。AIConfigurator 搜索的是多框架 serving 的配置组合，报告 648.3 tokens/s/GPU[c-object-serving]；Ada-MK 发现 kernel 启动开销占端到端推理时间的约 14.6%，于是把多个 kernel 融成 megakernel[c-object-serving]；RaMP 用四参数波形代价模型在运行时选择 MoE 的 dispatch 配置，相对穷举搜索的平均 regret 是 0.93%[c-object-serving]。

这一层最关键的差异不在技术，而在**判据**：这里没有「数值等价」这种廉价的、二值的、无歧义的判据。正确性退化成「SLA 是否满足、输出是否一致」，而性能要在真实负载下才可测——真实负载不可重复，测量噪声因此比 kernel 层高一个量级。

2.3 图与量化层：为什么本文只做旁证

图编译与量化压缩同样在被自动化，但它们的主力方法仍以编译器工程与离线搜索为主，LLM 驱动的闭环证据相对稀薄。本文因此把它们列为旁证而非主体：不是它们不重要，而是**证据密度不足以支撑「改进率」层面的结论**。

2.4 分层带来的第一个结论

四层对象的判据强度是递减的：

判据越自动化、越廉价、越无歧义，循环越闭合。这是第 4 章「反馈强度」分级的第一块基石，也是本文解释「为什么几乎所有 RSI 叙事都发生在 kernel 层」的直接原因。

3 循环的四种形态

3.1 四种形态与它们的成本结构

「递归自我改进」在工程上有四种可区分的形态，区别在于**改进发生在哪里** [c-loop-forms]：

这四种形态常被混为一谈，但它们的收益曲线完全不同。单轮形态的天花板由模型先验决定；多轮形态的收益随轮次增长但很快被上下文长度拖累；RL 形态把循环内化进权重，推理时不再需要循环；脚手架形态不改权重，只改「怎么用模型」。

3.2 同一预算下的搜索策略差异

即使在同一形态内，搜索结构的影响也不小。STARK 在相同预算下比较了 MCTS、进化、贪心与 ε-贪心四种策略，结论是 ε-贪心最稳 [c-loop-forms]。KernelFoundry 则报告进化式搜索在两次迭代内就找到正确 kernel，十次迭代达到 7.9 倍 [c-loop-forms]。

这两个结论并不矛盾，但也不能互相印证：它们的预算定义、评测集与基线都不同——这正是第 5 章要处理的口径问题。

3.3 不动权重也能进步

本文认为最值得注意的一条，是脚手架形态的效应量。

KernelBlaster 的消融给出一个干净的数：把长期记忆去掉、但保留完整的 Nsight Compute 分析能力，系统仍然显著变慢——`no_mem_agent` 的结果比完整系统慢 1.67 倍 [c-moat]。注意这里权重没变、反馈级别没变，变的只是「有没有把过去的经验取回来」。

RSIFlow 的消融同向：关掉 meta-agent 的演化，平均性能掉 5.74%[c-moat]。Self-HPO 则显示，把策略与脚手架一起更新，比只更新其中之一更好 [c-moat]。

把这些放在一起，得到的推论是：**这一代系统的「自我改进」，可迁移的资产是经验库与调度策略，不是模型能力。**这条主张本文标为 medium 而非 high——因为干净地「只动脚手架」的对照实验并不多，多数系统的脚手架与策略是一起变的 [c-moat]。

4 反馈信号：本文的主坐标

4.1 四级阶梯

把第 2 章的判据强度与第 3 章的循环形态合起来，可以得到一条更可操作的轴——**反馈强度**：

- **L1 仅编译通过**：知道代码能编译，不知道对不对、快不快。
- **L2 数值等价**：知道输出与参考一致。
- **L3 加速比**：知道相对基线快多少。
- **L4 语义诊断**：知道**为什么**慢——occupancy、访存合并、roofline 差距。

每一级都比前一级贵，也都比前一级更能指导搜索。而它们对改进率的影响是可以被单独测量的。

4.2 每一级的失效模式

L1 的失效是「能跑但不对」。在 Kernel-Smith 的观察里，模型会直接调用原生 PyTorch 算子来冒充「自己生成的高效 kernel」，从而伪造出约 1 倍的「通过」结果 [c-feedback-ladder]。这不是模型在偷懒，是 L1 这个信号本身不含正确性信息。

L2 的失效是「对但没用」。通过 `allclose` 的 kernel 完全可能比基线慢。STARK 的报告里，Reflexion 类 agent 的正确率可以到 100%，但产出的 kernel 比基线还慢 [c-loop-forms]。

L3 的失效是「快但作弊」。最典型的就是针对窄测试分布硬编码旁路 [c-feedback-ladder]。一旦评测输入的分布可知，优化目标就从「写出快的 kernel」退化成「猜中测试用例」。

L4 的失效是「贵」。采集 NCU 计数器要额外跑一遍，profiler 本身有开销，且返回的是大量原始指标——而 LLM 对长输入的检索精度会随长度下降。

4.3 阶梯是可测量的

这条阶梯不是本文的臆造，而是靠「同一系统、只换反馈」的对照实验立起来的：

- **L3 → L4**：KernelPro 报告，微基准形式的 profiler 工具比裸指标高出 **125%** 的加速比（$p < 0.0001$）[c-feedback-ladder]。
- **L2 → L3**：KernelBlaster 报告，只用标量延迟的 agent 在 Level-2 上是 1.22 倍，接入 NCU 数据后是 1.57 倍 [c-feedback-ladder]。
- **L1 → L2**：CUDAPerf 的消融显示，去掉可验证奖励后正确率掉 14%，去掉结构奖励后掉 8%[c-feedback-ladder]。

三个实验的方向一致，量级也一致：**反馈每上一个台阶，同一模型能换到的改进是两位数百分比级别的。**

表 2 四层对象的判据强度递减，「闭环」与否由判据的自动化程度决定

层	正确性判据	性能判据	闭环是否闭合
kernel / 算子	数值等价（可自动、秒级）	单 kernel 计时（低噪声）	闭合
图 / 编译	与参考实现等价	端到端吞吐	半闭合
服务与调度	SLA、输出一致	真实负载下的吞吐/延迟	半闭合
量化 / 压缩	精度回归阈值	精度-吞吐前沿	半闭合

表 3 四种循环形态：改进发生在采样、轨迹、权重还是脚手架

形态	改进发生在	主要成本	代表
单轮生成 + 验证	采样	一次生成	KernelBench 基线
多轮反思 / 修复	同一上下文内的轨迹	上下文与 token 线性增长	STARK、Debug-Agent
强化学习	模型权重	训练算力与奖励工程	CUDA Agent、CUDAPerf
脚手架自演化	harness / 记忆 / 搜索	经验库维护与迁移验证	KOPE、Self-HPO

4.4 验证器才是瓶颈

如果反馈强度是主坐标，那么系统的瓶颈就不在生成，而在**验证**——因为高强度的反馈需要真跑一遍。

这个判断被三组独立证据支持：

其一，验证预算可以被替换成更便宜的代理。「Verify Less, Evolve More」训练了一个 idea 级 critic，在把候选送进真实执行之前先预筛。结果是：只有 36.3% 的想法需要真验证，同预算下策略更新步数提高到 2.75 倍 [c-verifier]。

其二，修复比重生成便宜得多。 Debug-Agent 在 27 个 near-miss 算子上做到 Pass@1 66.7%，而三次重生成只有 40.7%，每次成功的 token 成本低 92.8%[c-verifier]。

其三，验证器必须能识别绕过。 Kernel-Smith 把 hacking 检测做成了流水线的一环 [c-verifier]，而 KernelBench-Verified 直接用隐藏测试集把这件事推到评测协议层面 [c-feedback-ladder]。

其四，验证对象可以前移到编译期。 Ave 用数据流不变量做护栏，把「这个优化是否违反全局约束」变成编译期就能给出反例的问题，从而完全避开了运行期验证的噪声 [c-feedback-ladder]。

生成不是瓶颈，验证是。这条判断有一个直接推论：任何声称「自我改进」的系统，如果它的验证器与训练/搜索用的是同一批测试用例，它的改进率就是被高估的。

5 增益：数字与口径

5.1 把数字并列

下表把本文核到的增益并列。每一行都标注了**相对什么基线、用什么统计口径**——不标这两项的数字不予采信：

跨度这么大，不是因为这些系统差十倍。看最后两行：ForgeKernel 的 +10% 是相对**专家参考实现**——基线本身就接近硬件峰值；案例研究的 9.293 倍，作者自己解释为「基线离各自峰值远近不同」，同一批 kernel 相对不同基线分别是 4.6 倍、14.0 倍和 21.3 倍 [c-gain]。

分母决定分子。这是本领域最重要的一条读表规则。图 1 把这七条并列，横轴是报告值，颜色区分是否达到 1.5 倍。

5.2 口径造成的差异与预算造成的差异同量级

KernelOPT 的报告很好地说明了这点：它在三个难度级上分别是 1.40、1.15、1.07 倍。同一系统、同一基线，仅因为题目难度不同，报告值就差 30%[c-gain]。而 AdaExplore 在预算从 50 步加到 100 步时，Level-2 从 2.65 升到 3.12 倍——约 18%[c-plateau]。

难度带来的差异（30%）大于预算带来的差异（18%）。这意味着「加预算」并不是提升报告值最有效的手段，「换一批更容易的题」才是——而这正是发布偏差最容易藏身的地方。

5.3 回报曲线是先陡后平

如果循环真的是「递归自我改进」，回报曲线应当出现加速。实测不是。

AdaExplore 是少见的把预算曲线完整画出来的工作：50 步到 100 步，Level-2 从 2.65 到 3.12 倍，Level-3 从 1.55 到 1.72 倍 [c-plateau]。还在涨，但斜率在变小。

ForgeKernel 的描述更直接：在同一架构内再调，只剩约 4.5% 的提升，之后「进一步的调整没有带来有效改进」[c-plateau]。而 CAKE 在昇腾 NPU 上跑满 20 轮预算时，仍未触及 roofline 停止阈值 [c-plateau]——说明它不是收敛了，是预算用完了。

这三条合起来说明：当前呈现的是边际递减的测

表 4 七条增益并列：同一层工作跨度接近一个数量级，基线各不相同

系统	报告值	基线	口径
KernelOPT L1/L2/L3	1.40 / 1.15 / 1.07	`torch.compile`	几何平均，250 题
Astra	1.32	SGLang 原实现	算术平均，3 个 kernel
KernelBlaster L1	1.43	eager 与 compile 取优	几何平均
AdaExplore L2 / L3	3.12 / 1.72	PyTorch	几何平均，100 步预算
cuPilot	3.09	PyTorch	算术平均，100 个 kernel
ForgeKernel	+10%	**专家参考实现**	几何平均，27 算子
案例研究	9.293	官方基线	几何平均

试时扩展，不是自我加速。图 3 把 AdaExplore 的两条曲线画在同一坐标系里，斜率下降是可算的：Level-2 前 50 步从 0 涨到 2.65 倍，后 50 步只从 2.65 涨到 3.12 倍（增量 0.47）。

5.4 kernel 级到端到端：一个稳定的漏斗

即使 kernel 层的增益是真的，它也不能直接当作「推理优化」的收益。本文核到五组「同一篇论文里同时给出 kernel 级与端到端两个数」的证据：

- RMSNorm kernel 快 6.65 倍，端到端只快 8.3%-10.1%[c-e2e]
- 子 kernel 快 3.27 倍，端到端降到 2.89 倍 [c-e2e]
- KForge 在 B200 上端到端只有 2.12%[c-e2e]
- Ada-MK 优化启动开销，端到端 23.6%（对 TRT-LLM）/ 50.2%（对 vLLM）[c-e2e]
- **最极端的一条**：kernel 级 5.84 倍的改进，端到端反而略慢——因为 Triton 的 dispatch 开销超过了融合带来的收益 [c-e2e]

kernel 级加速比是「局部指标」，端到端是「全局指标」，两者之间不是缩放关系，而是漏斗关系。而漏斗的宽窄由 Amdahl 定律与调度开销共同决定。图 4 给出三组同源对照，其中第三组的端到端柱低于 1.0。

6 代价：跑而不是想

6.1 时间去哪了

一个反直觉但稳定的测量：单次迭代里，**LLM 调用约占 21% 的总时间，编译-运行-采样等工具动作占 79%**[c-cost]。

这条数字改变了对「成本」的理解。直觉上，agentic 系统的开销在推理（token）；实测上，开销在**执行**。这也解释了为什么「减少迭代次数」比「换更便宜的模型」更能降低延迟——前者直接砍掉那 79%。

6.2 边际收益递减

如果成本主要在「跑」，一个自然的想法是「多跑几次」。实测不支持：

SpecGen 的对照显示，单纯把 CudaForge 的 token 预算翻倍，最终加速比只从 5.05 升到 5.15，从 2.61 升到 2.74[c-cost]。**预算翻倍，收益个位数百分比。**

而并行采样更贵：把候选数乘 K，token 成本也乘 K[c-cost]。这就是为什么「怎么花」比「花多少」更重要。

6.3 每次成功的成本

把成本按「每次成功」归一化，差异立刻显现：Debug-Agent 每次成功需要 0.371M uncached tokens，而重生成需要 1.678M——差 4.52 倍 [c-cost]。而要学会隐式策略（而非照着给定策略做），成本是后者的 4 到 101 倍 [c-cost]。

「学会」比「照做」贵两个数量级。这条数字对「RSI」的叙事是一个冷水：真正的自我改进需要从经验里提炼出可迁移的东西，而那恰恰是最贵的一步。图 2 把本章的时间拆分与 token 差画在一起——注意纵轴是对数轴，否则两者无法同图呈现。

6.4 一张最小报告清单

把第 5 章与本章放在一起，可以得到一张核对清单。本文建议任何声称「自我改进」的工作至少写清五项：

这五项之外，本文还建议给出**迭代曲线**而非单点：第 5 章的图 3 与第 7 章的四种失效共同说明，终点值相同的方法可能在曲线上完全不同——一个在第 20 轮还在涨，另一个在第 5 轮就平了。

这不是格式建议。下一章将说明，四类失效里有三类恰好对应这张表上被省略的某一项。

7 边界：四种失效

7.1 基线已接近峰值

KernelOPT 的失败分析给出一个干净的数：85 次回退中有 61 次（72%）是因为待优化的模型被

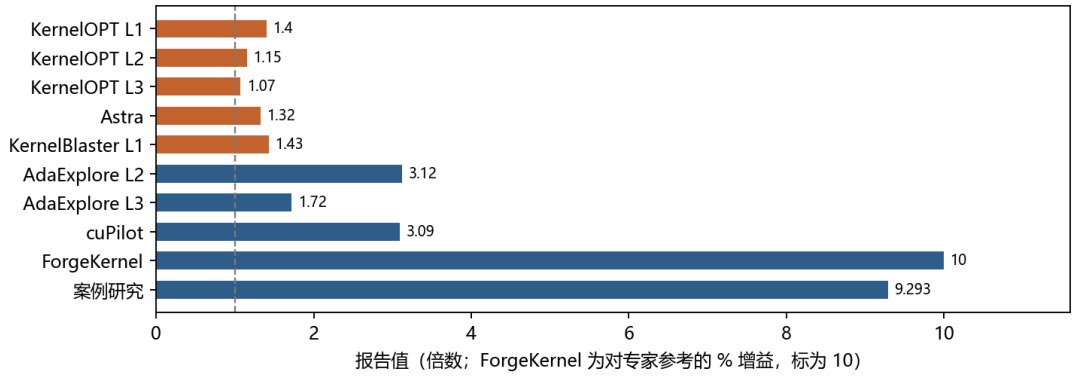


图 1　图 1　报告增益的跨度与口径差异。七条报告增益并列，同一层工作的报告值跨度接近一个数量级，其中 ForgeKernel 的 +10% 是对专家参考实现而非 PyTorch，案例研究的 9.293 倍则因基线距峰值远近不同而被作者自己打折。

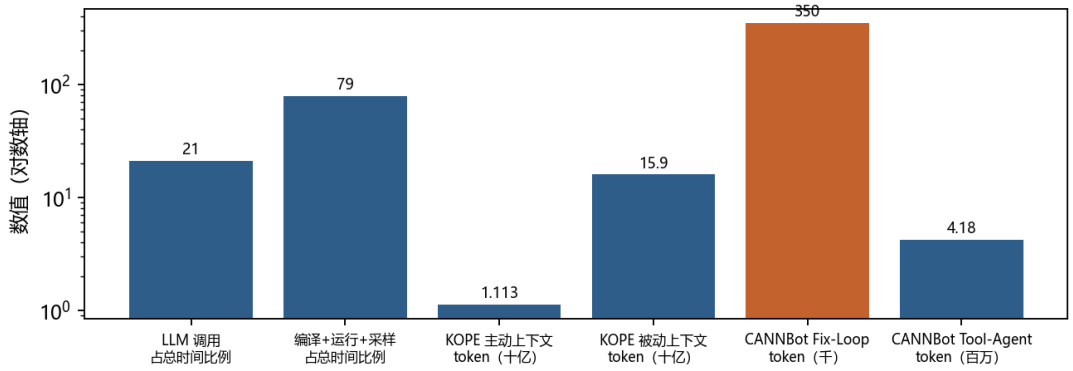


图 2　图 2　成本结构：跑而不是想。左半是单次迭代里时间的去向（LLM 调用 21%、编译-运行-采样 79%）；右半是同一任务下不同策略的 token 用量，相差两个数量级。两者共同说明：成本在「跑」，而「怎么花」比「花多少」更重要。

cuBLAS GEMM 或 cuDNN 卷积主导，而这些库调用在 H200 上已接近峰值 [c-boundary]。在 37 个 GEMM 主导的案例里，Triton epilogue 只占端到端时间的不到 1%，优化它反而因 dispatch 开销让整体变慢 [c-object-kernel]。

当分母已经是硬件峰值时，分子再怎么优化都不动。

7.2 分布偏移

KernelEvolve 在 480 个算子-平台配置上做到 100% 正确，生产负载 1.25-17 倍 [c-gain]；但在分布外形状上，同一批 kernel 掉到 **0.49-0.63 倍**——即比基线还慢 [c-gain]。

这不是缺陷，是设计：优化本来就针对生产分布。但它意味着「RSI 产出的 kernel」是**专用件**，不是通用件。部署时换一个 batch size，收益可能反号。

7.3 测量落后于模型

这是本文认为最危险的一种失效，因为它不制造错误，只制造**看起来对的错误**。

证据链是完整的：KernelBench 标准协议下 1.43 倍 → 收紧基线并加隐藏测试集后 0.88 倍 [c-feedback-ladder]；模型会硬编码旁路 [c-feedback-ladder]；会直接调 PyTorch 冒充 kernel[c-feedback-ladder]。

而缓解手段是存在的：AutoTuneBench 专门研究「agent 自动调 serving 引擎」的可信测量 [c-measure]，AgentKernelArena 研究泛化感知的评测 [c-measure]。

评测协议必须与模型能力共同演化——否则「改进率」测的是协议的老化速度。

7.4 改动本身无效

最容易被忽略的一种失效：在脚手架层做的改动，多数没有效果。

一项针对编码 agent 的实证研究追踪了 125 条 harness 修改提议，实现了 24 条，并在后续任务上评估了 12 条。在有可比性的 7 次评估中：2 次消除了目标行为、1 次减少、**4 次没有任何减少**，还有一些改动在无关问题上增加了工作量 [c-boundary]。

同一个方向的另一条证据：跨模型协同编译在基线利用率较低时收益显著（几何平均每轮 1.37 倍），但基线利用率越高，收益越小 [c-boundary]。

「自演化」的提议生成能力，远强于「判断一个

项	为什么必须写	本文对应
基线是什么	分母决定分子，专家实现与 PyTorch 相差一个数量级	表 4
统计口径	几何平均与算术平均在长尾分布下差别显著	表 4
反馈级别	L1-L4 决定改进的天花板与作弊风险	第 4 章
预算与预算曲线	只报终点值无法判断是否饱和	图 3
端到端效果	kernel 级增益与端到端不是缩放关系	图 4

改动是否真的有用」的能力。这与 4.4 节的结论同构：瓶颈永远在验证侧。

7.5 四种失效的共同结构

把四节并成一张表：

这张表揭示了一个共同结构：**四种失效里有三种，问题都不在搜索算法，而在「什么算改进」这件事没有被定义清楚。**这与第 4 章的结论（瓶颈在验证）是同一件事的两种说法。

8 讨论：这是 RSI 吗？

8.1 三条判据

把「递归自我改进」拆成可检验的三条：

1. **改进是否随时间加速？**若是，回报曲线应上凸。
2. **改进是否作用于改进者自身？**若只作用于被优化的对象，那是自动优化，不是自我改进。
3. **改进是否可累积、可迁移？**若每次从零开始，那是测试时扩展。

逐条对照本文证据：

- 第 1 条：**否**。回报曲线是下凹的 [c-plateau]。
- 第 2 条：**部分是**。脚手架层的自演化确实在改「改进者自身」的一部分（上下文构造、搜索策略、记忆），但改的不是产出改进的那个模型 [c-moat]。
- 第 3 条：**是，但有限**。记忆与经验库可以跨任务复用（去掉记忆慢 1.67 倍）[c-moat]，但跨硬件/跨分布的迁移会失效（分布外掉到 0.49-0.63 倍）[c-gain]。

结论：当前是「有累积性的自动优化」，不是「递归自我改进」。

8.2 为什么这个词仍然有用

如果严格意义上不成立，为什么这个说法还在扩散？因为它指向了一个真实的变化：**改进的瓶颈从「想不出办法」转移到了「验证不了办法」。**

在 LLM 进入之前，kernel 优化的瓶颈是知识（要知道 tiling、流水、bank conflict）。现在，这些知识模型大体都有，瓶颈变成了「怎么廉价而可靠地知道一个改动是不是真的更好」。所有本文核到的方法学创新——critic 预筛 [c-verifier]、修复代替重生成 [c-verifier]、不变量作编译期护栏、语义化 profiler 反馈 [c-feedback-ladder]——都在回答同一个问题。

这也解释了为什么这条曲线会饱和：验证的成本随候选质量提升而上升，而验证预算是有上限的。

8.3 四条可迁移的做法

撇开命名之争，本文认为有四个做法值得迁移到别的优化对象上：

1. **把反馈分级，并写进报告。**任何「改进率」都必须标注它拿到的是 L1-L4 中的哪一级 [c-feedback-ladder]。
2. **把验证成本当作一等公民。**优先做 critic 预筛与修复，而不是加大采样 [c-verifier]。
3. **同时报告 kernel 级与端到端。**缺少后者的加速比不构成推理优化的证据 [c-e2e]。
4. **在隐藏分布上复测。**分布内的成绩是上界，分布外的成绩才决定能否部署 [c-boundary][c-gain]。

9 结论

本文用 57 篇论文、74 条可回溯证据回答了开头那个矛盾：**同一批模型既能做到 1.43 倍，也会掉到 0.88 倍，差别不在模型，在反馈。**

由此得到四条结论：

第一，只有 kernel 层的循环真正闭合。因为只有这一层的正确性与性能判据都能被程序廉价、无歧义地判定 [c-object-kernel][c-object-serving]。

第二，反馈强度是解释改进率的主坐标。从「只编译」到「语义诊断」，每上一个台阶，同一模型能换到的改进是两位数百分比级别；而低级别反馈的系统性风险是被钻空子，不是被改进 [c-feedback-ladder]。

第三，增益的数字不可直接比较，代价的结构却高度一致。报告值跨度接近一个数量级且分母决定分子；而成本几乎全在执行侧（79%），且对预算的响应是次线性的 [c-gain][c-cost]。

第四，回报曲线是先陡后平，因此当前不是递归自我改进。真正在累积的是脚手架层的经验，而不是模型能力；而脚手架改动的有效率本身很低（7 次评估中 4 次无效）[c-plateau][c-moat][c-boundary]。

表 5　四种失效条件：前两条是「优化对象选错了」，后两条是「判据不够硬」

失效条件	机制	本文证据
基线已接近峰值	库调用占端到端主导，优化余量被 Amdahl 吃掉	85 次回退中 61 次源于此
分布偏移	优化针对生产形状，换形状后收益反号	分布外掉到 0.49-0.63 倍
测量落后	评测协议比模型进步慢，可被钻空子	收紧协议后 1.43 → 0.88 倍
改动无效	提议生成能力远强于效果判定能力	7 次可评估改动中 4 次无减少

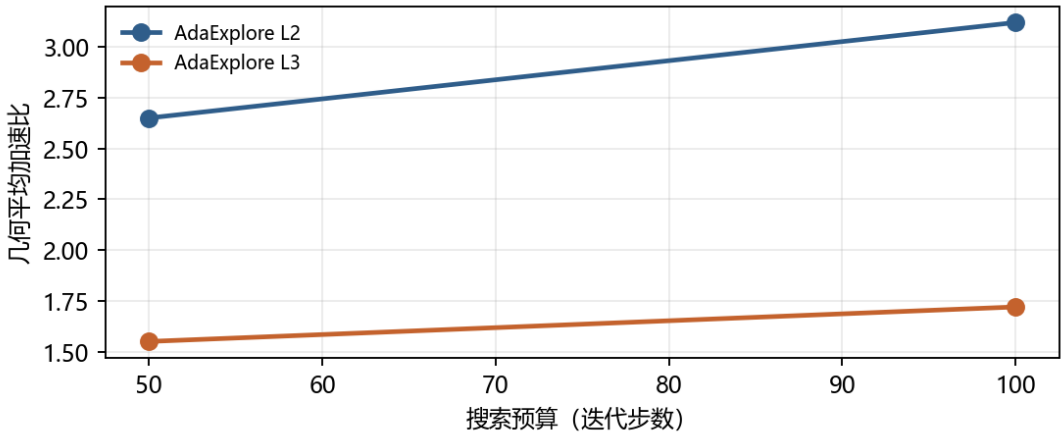


图 3　图 3　回报曲线：先陡后平。预算翻倍仍在涨，但斜率明显变缓。AdaExplore 是少数把预算曲线完整画出来的工作，其 Level-2 与 Level-3 的斜率都在下降。

如果要用一句话概括：**这一代系统没有让改进变快，它们让改进变得可以自动化——而自动化之后暴露出来的瓶颈，是验证。**

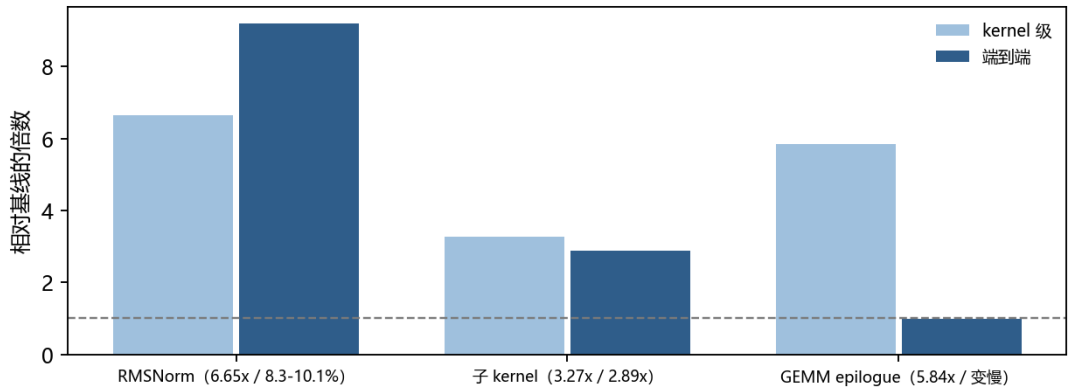


图 4　图 4　kernel 级到端到端：逐级衰减，乃至反号。三组来自同一篇论文内部的对照：RMSNorm、子 kernel、GEMM epilogue。第三组的端到端值低于 1.0，即 kernel 变快而模型变慢。

参考文献

下列 40 个来源被正文实际引用；归档共 57 个，其余为检索与筛选过程的记录。完整标题见 `sources.csv`。

[1] GPU Kernel Scientist: An LLM-Driven Fr… 2506.20807
[2] Astra: A Multi-Agent System for GPU Ke… 2509.07506
[3] STARK: Strategic Team of Agents for Re… 2510.16996
[4] cuPilot: A Strategy-Coordinated Multi… 2512.16465
[5] KernelEvolve: Scaling Agentic Kernel C… 2512.23236
[6] CUDA Agent: Large-Scale Agentic RL for… 2602.24286
[7] Kernel-Smith: A Unified Recipe for Evo… 2603.28342
[8] KernelFoundry: Hardware-aware evolutio… 2603.12440
[9] AdaExplore: Failure-Driven Adaptation… 2604.16625
[10] Ave: Guiding Agentic GPU Optimization… 2604.18616
[11] SpecGen: Accelerating Agentic Kernel O… 2606.17518
[12] Beyond Scaling: Self-Evolving LLM Agen… 2608.25570
[13] KernelOPT: Dispatch-Aware Agentic Sear… 2609.30059
[14] Ask, Don’t Equip: Question-Driven Codi… 2611.03517
[15] ForgeKernel: Self-Evolving Kernels Cat… 2611.04998
[16] Optimizing CUDA like a Human: Micro-Pr… 2606.26453
[17] Improving Efficiency of GPU Kernel Opt… 2603.29010
[18] Don’t Regenerate, Debug: A Domain-Spec… 2608.02712
[19] AsmEvo: Agentic Assembly-Level Optimiz… 2608.20711
[20] CANNBot-CAKE: Collaborative Agentic Ke… 2610.31034
[21] AgenticCANN: Automated Ascend C Operat… 2607.26661
[22] KForge: LLM-Driven Cross-Platform Kern… 2606.02963
[23] A Case Study of a Multi-Agent Kernel-O… 2610.38329
[24] Multi-turn RL with Structural and Perf… 2607.20908
[25] KernelBlaster: Continual Cross-Task CU… 2602.14293
[26] Afterburner: Reinforcement Learning Fa… 2505.23387
[27] ServeLearnBench: How Well Can Agents S… 2610.07792
[28] Conjugate Compilation: Self-Improving… 2610.25012
[29] Ada-MK: Adaptive MegaKernel Optimizati… 2605.11581
[30] RaMP: Runtime-Aware Megakernel Polymor… 2604.26039
[31] AIConfigurator: Lightning-Fast Configu… 2601.06288
[32] Improving Coherence and Persistence in… 2603.21321
[33] RSIFlow: Adaptive Self-Improvement Wor… 2610.21791
[34] Verify Less, Evolve More: Training Ide… 2610.08993
[35] Self-Evolving Coding Agents via Harnes… 2610.22791
[36] How Do Coding-Agent Harnesses Self-Evo… 2610.34938
[37] KernelBench: Can LLMs Write Efficient… 2502.10517
[38] KernelBench-Verified: Do LLM-Generated… 2607.16241
[39] AgentKernelArena: Generalization-Aware… 2605.16819
[40] AutoTuneBench: Trustworthy Measurement… 2609.18123